

From Attack Behavior to Defense Failure: A Stage-Aware Framework for Evaluating APT Defense Capability

Anonymous Author(s)

Abstract

CCS Concepts

• **Computing methodologies** → **Multi-agent systems**; • **Software and its engineering** → *Software creation and management*.

Keywords

APT defense evaluation, control-point attribution, stage-aware scoring, security benchmark

ACM Reference Format:

Anonymous Author(s). 2026. From Attack Behavior to Defense Failure: A Stage-Aware Framework for Evaluating APT Defense Capability. In *Proceedings of THE ACM WEB CONFERENCE 2026* (Conference acronym 'WWW). ACM, New York, NY, USA, 11 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Advanced Persistent Threats (APTs) have become a major threat to national, enterprise, and critical-infrastructure cybersecurity. According to M-Trends 2026, the global median dwell time rose to 14 days in 2025, while the median dwell time for cyber-espionage intrusions reached 122 days[?]. Verizon DBIR 2025 reports that credential abuse and vulnerability exploitation remain major initial attack vectors, and that the share of breaches involving third parties doubled to 30%[?]. IBM's 2024 Cost of a Data Breach report further shows that the global average cost of a data breach reached \$4.88 million, and that about 70% of affected organizations experienced significant or moderate business disruption[?]. Taken together, these reports suggest that APT defense evaluation should focus not only on whether an intrusion is eventually found, but also on where defensive control is lost during a multi-stage campaign.

In response, organizations deploy layered defense stacks, including Endpoint Detection and Response (EDR), Next-Generation Firewalls (NGFW), Security Information and Event Management (SIEM), Privileged Access Management (PAM), and email security gateways. However, deployment does not imply effectiveness. A defense stack with more products is not necessarily more capable, and a system that fails to stop an entire APT campaign is not necessarily useless. What matters is whether the evaluation can show where the defense worked, where it failed, and which failures should be repaired first.

Existing evaluation approaches provide useful pieces of this picture, but they do not directly produce repair-oriented, stage-aware

defense capability scores. Attack knowledge bases such as MITRE ATT&CK provide a common language for describing attacker behavior, and ATT&CK Evaluations report product responses at the step level[?]. Vulnerability scoring and attack-graph methods analyze severity, reachability, or path risk[? ? ?]. These approaches are valuable, but they usually stop at attack techniques, product observations, vulnerabilities, or attack paths. They do not directly answer which victim-side control point first failed for a missed action, nor how that failure should affect a multi-stage APT defense score.

This paper starts from a different evaluation unit: an APT action paired with its observed defense result and its first-failed victim-side control point. The motivation is simple. The same ATT&CK technique may require different repairs in different contexts. A credential-related action, for example, may result from weak password policy, missing multi-factor authentication, exposed remote login, insufficient credential storage protection, or successful phishing. These cases may share an attack label, but they do not point to the same defensive repair. A useful scoring framework should therefore preserve the link between attacker behavior, defense observation, and repairable control failure.

Building such a framework raises three practical challenges. First, the attribution schema must be stable enough to handle heterogeneous APT actions while remaining useful for repair. Second, the attack-stage dimension must be represented separately from control-point domains, since the same type of defensive failure may appear at different phases of an attack. Third, action-level defense observations must be aggregated according to stage logic rather than by a simple average, while keeping score loss traceable after aggregation.

This paper proposes a stage-aware APT defense capability scoring framework based on control-point attribution. The framework first maps each in-scope attack action to the victim-side control point that failed first and is most relevant for repair. It then enriches the attributed action with a canonical attack phase, expected defense product categories, confidence information, and stage aggregation semantics. Finally, it aligns defense observations with benchmark actions, assigns action-level hit values, and aggregates them into playbook-level and multi-dimensional defense scores.

The main contributions of this paper are as follows:

- We propose a control-point attribution schema for APT defense evaluation, shifting the evaluation unit from “what the attacker did” to “which victim-side control failed first.” The schema covers 9 top-level domains and 94 leaf-level control points, enabling action-level attribution of defensive failures.
- We design a benchmark construction process that connects control-point attribution with stage-aware scoring. It adds

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'WWW, Dubai, United Arab Emirates
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/18/06
<https://doi.org/XXXXXXX.XXXXXXX>

canonical phase mapping, expected defense categories, confidence labels, and stage aggregation semantics to produce scoring-ready defense opportunity records.

- We develop a stage-aware scoring model that separates action-level hit values from stage importance and supports aggregation from actions to stages and playbooks. The model also reports scores by attack phase, control-point domain, and defense product category, making score loss easier to interpret.
- We evaluate the framework on 53 APT playbooks containing 1,758 attack actions. The results validate the mathematical behavior of the scoring engine and show that no single product category can independently cover the full APT attack surface, highlighting the need for coordinated defense-in-depth.

2 Related Work

Prior work supplies several components needed for APT defense evaluation, but these components use different evidence and units of analysis. Attack knowledge bases describe adversary behavior; detection systems transform telemetry into alerts or attack reconstructions; network-enforcement systems implement and evaluate specific controls; and vulnerability or attack-graph metrics estimate structural risk. We organize the literature around these roles and distinguish their outputs from the action–outcome–control tuple scored in this paper.

2.1 Attack Knowledge Bases and Adversary-Emulation Evaluation

MITRE ATT&CK provides the common tactic and technique vocabulary used to normalize adversary behavior in our benchmark [?]. ATT&CK Evaluations add a different kind of evidence: they record how products expose or detect individual procedures during adversary emulation [?]. These step-level observations are a natural input to a scoring framework, but they remain observations until they are aligned with benchmark actions and assigned a consistent outcome.

D3FEND organizes cybersecurity countermeasures and their relationships to adversary behaviors [? ?]. Its role is therefore prospective: given an adversary behavior, it identifies defensive techniques that may be relevant. Our attribution layer addresses the reverse, evidence-driven question. Given an action and an observed defense result, it identifies the victim-side control point that failed first and is most relevant for repair. D3FEND can inform the defense-technology crosswalk, but a capability relationship alone does not establish that a deployed system prevented, blocked, detected, or missed an action.

Academic work has also examined how attack knowledge is extracted and represented. iACE extracts indicators and contextual relations from open-source threat reports [18], while ATTACK2VEC learns temporal relationships among attack steps observed in large-scale security events [24]. Li et al. show that operational threat-intelligence feeds differ substantially in coverage and timeliness [17], motivating explicit provenance for benchmark mappings rather than treating all threat descriptions as equivalent evidence.

A separate line connects high-level attack descriptions to system evidence. POIROT converts threat-intelligence descriptions into query graphs and aligns them with kernel-audit provenance through inexact graph matching [21]. ATLAS instead learns sequences of attack symptoms and constructs attack stories for investigation [1]. These systems narrow the semantic gap between an attack description and low-level observations. Their output, however, is matched activity or an attack reconstruction; they do not define how heterogeneous observations should be normalized into defense outcomes or aggregated into repair-oriented scores.

2.2 APT Detection, Correlation, and Attack Investigation

Early provenance systems relied on explicit causal or behavioral structure. SLEUTH propagates tags through commodity audit events to reconstruct causal attack scenarios in real time [13]. HOLMES correlates suspicious information flows into higher-level APT scenarios using knowledge of adversarial behaviors [22]. Both preserve multi-step context, but their primary result is an attack graph or campaign-level detection rather than a measure of how well a defense stack handled each benchmark action.

Other systems focus on reducing investigation cost or improving evidence fidelity. NoDoze combines provenance context with anomaly scoring to prioritize alerts and reduce the graphs presented to analysts [10]. OmegaLog correlates application and system logs so that investigators can recover higher-fidelity attack provenance without relying on either log layer alone [11]. Their contribution is not simply “better detection”: NoDoze addresses triage, whereas OmegaLog addresses cross-layer evidence reconstruction.

Learning-based approaches use different representations and output granularities. DeepLog models ordered log events with recurrent neural networks and reports anomalous execution sequences [6]. Log2Vec constructs heterogeneous enterprise-log graphs and learns embeddings for threat detection [19]. These systems operate on log sequences or entities, whereas the scoring unit in this paper is a benchmark action aligned with a defense observation.

Within provenance-based detection, UNICORN summarizes long-running system provenance and learns unsupervised models for low-and-slow APT behavior [8]. Tactical provenance analysis raises endpoint events to behavior-level abstractions useful to EDR investigation [9], while threaTrace learns node-level representations to detect and trace suspicious host entities [28]. These differences matter because an anomaly, a tactic-level abstraction, and a suspicious node are not interchangeable observations; each requires a separate alignment rule before it can support action-level scoring.

Recent work has concentrated on deployment scale, online processing, and interpretable output. DISTDET distributes detection across hosts and performs higher-level correlation for large enterprise deployments [5]. PROGRAPHER limits provenance dependency expansion by embedding time-ordered graph snapshots [30]. NODLINK formulates online provenance-graph construction as an online Steiner Tree Problem and uses an approximation algorithm to recover concise, fine-grained attack graphs [16]. TAPAS instead segments process provenance by task to reduce the spatial and temporal cost of long-running online detection [33]. The four systems therefore address different deployment constraints: distributed

correlation, dependency control, concise online investigation, and long-duration graph segmentation.

Representation-learning systems make another set of tradeoffs. MAGIC uses masked graph representation learning for entity- and batch-level detection and includes adaptation for concept drift [14]. KAIROS learns the temporal evolution of whole-system provenance, assigns fine-grained event anomaly scores, and reconstructs compact attack summaries [3]. FLASH also learns provenance representations for scalable intrusion detection, but uses a different semantic and structural representation design [23]. R-CAID is particularly relevant to our attribution discussion because it incorporates root-cause analysis into provenance-based detection [7]. Its output identifies the causal origin of suspicious activity in the observed provenance. Our first-failed control point instead identifies the defensive mechanism or policy that should be repaired. The two forms of explanation are complementary but not equivalent.

Collectively, detection and investigation systems can supply alerts, suspicious entities, compact subgraphs, attack stories, or causal roots. Our framework treats these artifacts as possible defense observations. It adds an evaluation layer that aligns them with benchmark actions, normalizes them into PREVENTED, BLOCKED, DETECTED, or MISSED, and retains a link from score loss to a repairable victim-side control point.

2.3 Network Enforcement and Defense-System Evaluation

Network-enforcement research provides examples of controls whose effects can be observed directly during replay. SANE centralizes enterprise connectivity decisions in a protected network architecture [2]; its contribution is architectural prevention rather than post hoc detection. FIREMAN analyzes firewall rules and topology to identify conflicts and policy misconfigurations before deployment [32]. These works illustrate two different evidence types: an enforced communication decision and a static finding about policy correctness.

Software-defined networking introduced mechanisms for both active defense and systematic assessment. AVANT-GUARD moves connection management and event triggers toward OpenFlow switches to protect the controller and enable responsive enforcement [26]. SPHINX monitors evolving SDN state and detects behavior inconsistent with the expected network model [4]. DELTA takes an assessment-oriented approach, orchestrating known attacks and protocol-aware fuzzing against SDN controllers and applications [15]. AVANT-GUARD and SPHINX are defenses that can emit blocking or detection evidence; DELTA is closer to the replay infrastructure that generates such evidence.

Programmable data planes support higher-rate and more specialized enforcement. Poseidon implements configurable volumetric-DDoS detection and mitigation in programmable switches [34]. Jaqen builds switch-native detection primitives and network-wide resource management for dynamic high-rate attacks [20]. Ripple targets adaptive link-flooding through decentralized, declaratively programmed switch defenses [29]. SmartCookie combines programmable-switch SYN-cookie checks with an eBPF proxy to block SYN floods while preserving benign-service performance [31].

Their evaluations are not directly interchangeable because the systems differ in attack model, enforcement location, platform requirements, and reported output.

Such systems provide the kind of empirical evidence required by our scoring layer: a flow may be impossible by design, dropped during execution, observed without interruption, or missed. Conventional evaluations report mechanism-specific quantities such as detection accuracy, blocking rate, throughput, latency, and overhead. Our framework does not replace those measurements. It aligns them with playbook actions and aggregates the resulting outcomes across stages and attack paths.

The full attribution schema in this paper is broader than network enforcement: it also covers endpoint, identity, data, and operational controls. A Mininet-based experiment therefore constitutes a network-replayable validation slice rather than experimental coverage of the complete taxonomy. Actions that require unsupported endpoint state must be reported as out of scope or explicit experimental preconditions, not as defense failures.

2.4 Vulnerability, Attack-Graph, and Defense-Effectiveness Metrics

Vulnerability scores and attack graphs quantify security using evidence that differs from runtime defense observations. CVSS estimates the severity of an individual vulnerability from exploitability, impact, and environmental factors [? ?]. Its unit is the vulnerability, not an adversary action together with a product response and a failed control.

Attack-graph methods reason over configurations, privileges, and reachability. Sheyner et al. generate attack graphs through symbolic model checking [25]. MulVAL derives attack paths from logical facts about hosts, vulnerabilities, and connectivity [?], and later work improves attack-graph generation at larger network scale [?]. These methods preserve path dependencies that simple vulnerability counts miss, but the paths describe what may be reachable under a model rather than what a deployed defense actually observed during replay.

Security metrics extend this structural view. The k -zero-day safety metric measures how many unknown vulnerabilities an attacker would require to compromise a target [27]. Network-diversity metrics estimate resilience arising from heterogeneous software and configurations [35]. Both are useful structural measures, but neither should be reported as empirical detection or blocking performance. More broadly, Herley and van Oorschot argue that security claims require explicit assumptions, evidence, and limits of inference [12]. Following this principle, our reporting separates *structural applicability*, derived from the defense crosswalk, from *empirical defense performance*, derived from observed replay traces.

2.5 Synthesis and Positioning

Table 1 summarizes the distinction across research lines. The proposed framework does not claim to replace attack vocabularies, provenance detectors, network controls, or attack-graph metrics. Simply combining their outputs is insufficient because they use incompatible evidence semantics and units of analysis: an alert, a

suspicious subgraph, a blocking decision, and a prospective path-risk value do not identify the same evaluated event and cannot be aggregated without an explicit alignment rule. Our framework supplies that missing layer by mapping heterogeneous observations to benchmark actions, normalizing their outcomes, preserving stage logic, and attributing score loss to a first-failed victim-side control point. The framework and full benchmark remain broader than the network testbed; the planned Mininet evaluation is an empirical validation slice over network-replayable actions rather than experimental coverage of the complete taxonomy. This action–outcome–control unit permits stage-aware aggregation while keeping an aggregate loss traceable to a concrete remediation target.

3 Background and Problem Statement

This section defines the evaluation setting considered in this paper. The goal is not to provide a general introduction to APTs, but to clarify the objects that are scored by our framework: APT playbook actions, defense observations, attack stages, and victim-side control-point failures.

3.1 APT Playbooks, Actions, and Stages

An APT campaign is usually described as a multi-stage process rather than as a single attack event. In this paper, we represent such a process using an APT playbook. A playbook contains a sequence of stages, and each stage contains one or more attack actions. An attack action is the smallest unit that can be aligned with a defense observation and later used for scoring.

This action-level view is important because a playbook-level result alone is too coarse. A defense system may fail to stop an entire campaign, but still detect or block important actions along the way. Conversely, a system may generate many alerts while still missing the action that allows the attacker to move to the next stage. For this reason, the benchmark should preserve action-level evidence before aggregating results to stages and playbooks.

Stages also differ in how their actions contribute to attacker progress. In some stages, several actions represent alternative ways to achieve the same objective; in others, multiple actions must be completed together; in still others, a group of related actions jointly describes an attacker activity such as discovery or collection. These differences mean that stage-level scoring should not treat every group of actions as a simple average. The exact aggregation rules are defined in Section 4.5; here, the key point is that stage structure is part of the evaluation problem.

3.2 Defense Observations and Evaluation Outcomes

A deployed defense stack may include endpoint, network, identity, email, logging, and response components. During an evaluation, these components may produce different kinds of evidence, such as prevention records, blocking events, alerts, logs, or analyst-facing detections. We refer to such evidence as defense observations.

A defense observation is not yet a score. It first needs to be aligned with a benchmark action. After alignment, the observation can be normalized into an action-level outcome. In this paper, we use four outcome levels: PREVENTED, BLOCKED, DETECTED, and MISSED. These levels distinguish actions that are removed by design,

interrupted at runtime, observed by the defense, or not covered by available evidence.

This distinction is needed because different outcomes have different defensive meanings. Preventing an action through architecture or policy is not the same as detecting it after execution. Blocking an action during runtime is also different from merely logging it. A scoring framework should therefore preserve the outcome level before aggregating action results into larger scores.

3.3 From Attack Behavior to Control-Point Failure

Attack descriptions and defense repair targets are not the same object. An attack technique describes what the attacker did. A control-point failure describes why the victim environment allowed the action to succeed or why the defense did not respond effectively. For repair-oriented evaluation, the second question is essential.

The same attack behavior may correspond to different defensive failures in different contexts. For example, credential-related activity may be associated with weak password policy, missing multi-factor authentication, exposed remote login services, insufficient credential storage protection, phishing exposure, or delayed response. These cases may look similar from the attacker’s side, but they imply different repair actions from the defender’s side.

We therefore use the notion of a victim-side control point. A control point is a defensive mechanism, configuration, policy, monitoring capability, or operational process that can be evaluated and improved. For each in-scope action, the evaluation should identify the control point that failed first and is most relevant for repair. This attribution does not replace attack-phase information. Instead, it complements it: the control point explains where the defense failed, while the attack phase explains when the action occurs in the campaign.

3.4 Problem Statement

The problem studied in this paper can be stated as follows. Given a set of APT playbooks and action-level defense observations, construct a scoring framework that measures defense capability against multi-stage APT behaviors while keeping the resulting scores interpretable and repair-oriented.

More specifically, the framework should satisfy three requirements.

- (1) **Repair-oriented attribution.** Each in-scope attack action should be connected to a victim-side control-point failure that can guide remediation. Actions that do not correspond to a directly repairable victim-side failure should be retained for playbook completeness, but excluded from primary attribution-based scoring unless a concrete victim-side control failure can be identified.
- (2) **Stage-aware aggregation.** Action-level outcomes should be aggregated according to the role of the action within its stage and the stage within the playbook. The framework should avoid treating all actions as equally valuable or all stages as having the same attacker-progress logic.
- (3) **Traceable score reporting.** Aggregated scores should remain explainable after computation. A low score should be traceable to the relevant playbook, stage, action, defense

Table 1: Positioning by evidence, unit of analysis, output, and remediation traceability.

Research line	Primary evidence	Unit of analysis	Typical output	Remediation traceability
Knowledge bases and emulation	Technique descriptions and emulation telemetry	Technique or emulated step	Vocabulary, mapping, or product observation	Partial; capability-oriented
Detection and investigation	Logs, alerts, and provenance graphs	Event, entity, subgraph, or attack story	Alert, suspicious graph, reconstruction, or causal root	Primarily attack-oriented
Network enforcement	Policies, packets, flows, and network state	Rule, flow, or attack instance	Policy finding, detection, or blocking decision	Mechanism-specific
Vulnerability and attack-graph metrics	Vulnerabilities, configuration, topology, and privileges	Vulnerability, configuration, or path	Severity, reachability, or structural resilience	Configuration-oriented
This work	Playbook action and aligned defense observation	Action–outcome–control tuple	Stage-aware, multi-dimensional defense score	Explicit first-failed control point

outcome, product category, and victim-side control-point domain.

These requirements motivate the methodology in the next section. The proposed framework first attributes attack actions to repairable control points, then enriches them with phase and defense-category information, aligns defense observations with benchmark actions, and finally computes stage-aware and multi-dimensional scores.

4 Methodology

4.1 Overview

This section describes how the proposed method turns APT playbooks and defense observations into scores that remain traceable after aggregation. The central requirement is traceability: a score should not be interpreted only as an aggregate number, but should remain connected to the playbook, stage, attack action, and first-failed victim-side control point that contributed to it.

Fig. 1 gives the overall workflow. The workflow has three parts. First, benchmark construction converts raw playbook actions into scoring-ready records. Each action is assigned a control-point attribution, a canonical attack phase, expected defense categories, a confidence label, and stage aggregation semantics. These fields form the defense opportunity map used by the scoring engine.

Second, defense scoring aligns observed defense results with the benchmark records. A tested system reports action-level outcomes, which are normalized into four hit levels: PREVENTED, BLOCKED, DETECTED, and MISSED. These hit levels capture whether the action was removed by design, interrupted at runtime, only observed, or missed.

Third, the scoring model aggregates the aligned hit values. The aggregation is stage-aware: OR stages, AND stages, and Cluster stages use different rules because they represent different attacker progress conditions. The framework then reports not only an overall score, but also phase-wise scores, control-point domain scores, product-category scores, and action-level attribution details.

4.2 Defensive Weakness Attribution

The first step is to view each attack action from the defender’s side. For every in-scope action, we identify the victim-side control point that failed first and is most relevant for repair. A control point may be a defensive mechanism, a configuration, a policy,

or an operational process that can be evaluated and remediated independently. A successful attack action therefore indicates that some victim-side control was absent, misconfigured, bypassed, or insufficiently enforced.

This scope excludes actions that do not correspond to a directly repairable victim-side failure. Pre-compromise reconnaissance, attacker-side infrastructure preparation, and similar external prerequisite activities are retained for playbook completeness, but they are not included in primary weakness statistics or attribution-based scoring unless a concrete victim-side control failure can be identified.

This attribution step is needed because an ATT&CK label tells us what the attacker did, but not which defense failed. The same ATT&CK technique may be enabled by different weaknesses in different environments. For example, a credential-related action may result from weak password policy, missing multi-factor authentication, exposed remote login services, insufficient credential storage protection, or successful phishing. These cases may share similar attack labels, but they imply different repair actions.

We therefore interpret each action by asking a repair-oriented question: if only one control point could be fixed, which one would most directly prevent or disrupt this action? The answer is organized through a structured control-point schema. This schema is not organized as an attack sequence. It separates defensive failures along four design dimensions: security function, architectural location, defensive lifecycle phase, and trust source. These dimensions are not intended to form a strictly orthogonal category system. They are used as practical design constraints for separating control points that lead to different repair actions.

Security function follows the long-standing view that protection mechanisms should be separated by their security roles, such as authentication, authorization, privilege separation, execution control, and data protection [?]. Architectural location captures where the relevant trust boundary or enforcement point sits, which is consistent with zero-trust architecture’s shift from static network perimeters toward users, assets, resources, and policy enforcement around them [?]. Defensive lifecycle phase distinguishes controls that prevent an action from succeeding from controls that detect, respond to, or recover from successful actions, aligning with the preventive–reactive distinction in cyber defense modeling [?]. Trust source captures whether the failed control is tied to externally supplied inputs, such as phishing messages, malicious documents,

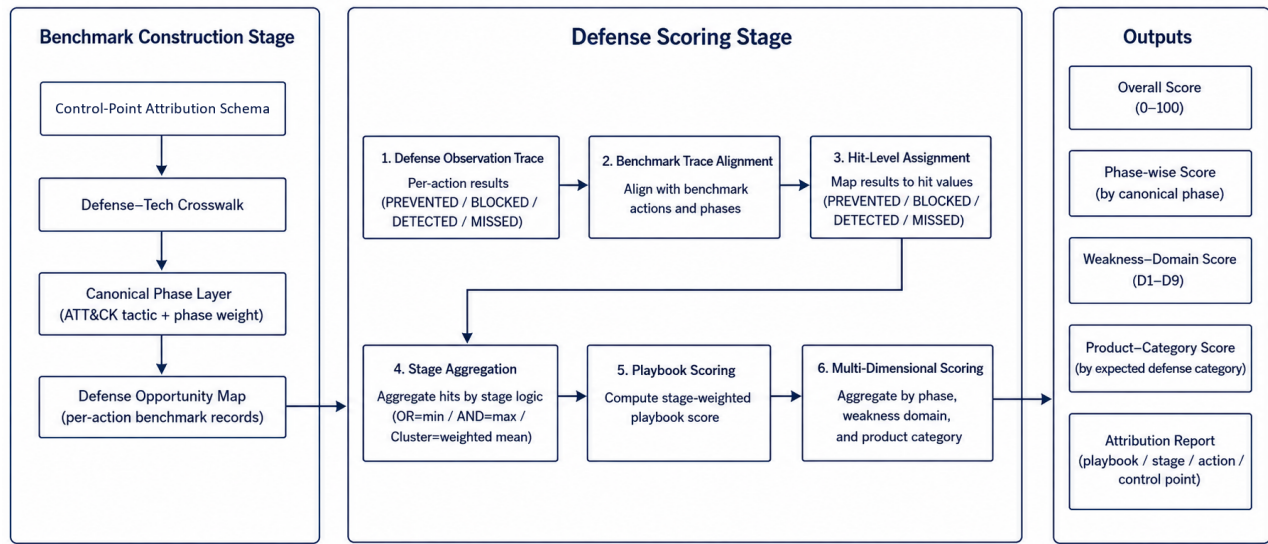


Figure 1: Workflow of the APT defense capability scoring framework.

Table 2: Top-level control-point domains.

Domain	Scope
D1	Identity, credential, and trust-root failures
D2	Authorization, isolation, and boundary-control failures
D3	Execution, host, and runtime-protection failures
D4	External exposure and remote-entry failures
D5	Communication and protocol-design failures
D6	Data protection, cryptographic-material, and recovery failures
D7	Observability, detection, and response failures
D8	External trust-input failures
D9	Availability and resilience failures

third-party collaboration, or supply-chain artifacts; this is related to attack-surface models that treat entry points, channels, and untrusted data items as key resources through which attackers interact with a system [?].

These dimensions also clarify why the nine domains are not arranged as an attack timeline. The domains describe different ways in which victim-side controls can fail, while the separate canonical phase layer captures when an action occurs in the attack chain. The schema contains nine top-level control-point domains and 94 fourth-level leaves. The domains indicate where score loss is concentrated, while the leaves identify the concrete controls that should be repaired.

Each in-scope action is assigned exactly one primary weakness leaf. This single-label rule keeps later scoring traceable. If one action had multiple primary attributions, the same failure could be counted several times, and score loss could not be linked to a clear remediation target. To keep attribution consistent, the schema uses

explicit boundary rules and the repair-oriented decision principle described above.

Some actions still involve more than one relevant weakness. Secondary weakness tags record such supporting conditions, including dual-control dependencies and entry-root-cause splits. These tags help explain attack chains, but they are not counted as primary attribution results and do not replace the primary leaf used for scoring.

After this step, raw attack actions have been converted into repair-oriented control-point failures. This attributed action set is the basis for canonical phase assignment, defense opportunity mapping, and stage-aware scoring.

4.3 Benchmark Construction

Control-point attribution identifies which victim-side control point failed, but it is not enough for scoring by itself. The scoring engine also needs to know when the action occurs in the attack chain, which defense product categories are expected to address it, and how the action should be aggregated with other actions in the same stage. Benchmark construction adds this information without changing the primary attribution result.

The process has three parts. Attributed actions are first assigned canonical attack phases using ATT&CK. Their control-point leaves are then linked to expected defense product categories. Finally, the enriched records are written into the defense opportunity map, which serves as the direct input to defense scoring.

4.3.1 Canonical Phase Assignment. The attribution domains describe where the victim-side defense failed; they deliberately do not encode when the action occurs in the attack chain. This temporal information is still needed for scoring and reporting. It allows the framework to distinguish defense performance across attack phases, apply phase-aware weights, and interpret stage-level results in relation to attacker progress. For example, interrupting an

early action usually has more defensive value than observing a late action after the attacker has already progressed through several stages. We therefore add a separate canonical phase layer.

For each action, the framework assigns a canonical phase based on its ATT&CK technique. We use ATT&CK Enterprise tactics as the standard phase vocabulary. Some ATT&CK techniques can be associated with multiple tactics. We resolve this ambiguity using a predefined technique-to-phase mapping. Each technique is mapped to a single canonical phase before scoring, and this mapping remains fixed during defense scoring.

Stage-level phases are then derived from the action-level phases, for example by majority voting among the actions in the same playbook stage. This phase assignment is an additional benchmark field; it does not change the control-point attribution described in the previous subsection. The canonical phase field is later used by the scoring model to apply phase weights. The main intuition is that earlier interruption has higher defensive value, while late-stage detection provides less benefit because the attacker has already progressed through much of the playbook.

4.3.2 Defense-Category Mapping. The primary weakness leaf identifies the failed control point. A scoring benchmark also needs to know which type of defense product should be expected to address that failure. For this purpose, we build a mapping from control-point leaves to defense product categories.

Each fourth-level leaf is mapped to two sets of defense categories. The first set is the primary defense category set. It contains product types that should directly prevent or block the corresponding weakness if deployed and configured correctly. The second set is the compensating defense category set. It contains product types that may detect, mitigate, or partially compensate for the weakness, but are not the most direct repair target.

For example, a brute-force action attributed to weak password and anti-brute-force policy is primarily related to identity and access management, while privileged access management or log monitoring may provide compensating support. Similarly, a C2 communication action attributed to missing egress proxy enforcement may be primarily related to network access control or next-generation firewall capabilities, while SIEM or IDS may provide secondary visibility.

This mapping allows later scoring to evaluate each product category within its expected coverage set. A product is therefore not judged against all benchmark actions equally, but against the actions that its category is expected to cover.

4.3.3 Defense Opportunity Map. The final output of benchmark construction is the defense opportunity map. Each record in the map corresponds to one attack action and combines the information needed for scoring: the playbook identifier, stage number, action description, ATT&CK technique, primary control-point leaf, top-level domain, canonical phase, expected defense categories, confidence label, and stage aggregation semantics.

The opportunity map keeps the primary attribution unique, but it can also record additional defense opportunities when they are available. In particular, secondary weakness tags can be expanded into additional leaves and defense categories. This preserves the repair-oriented primary attribution while still representing layered defense opportunities.

Table 3: Main fields in a defense opportunity record.

Field	Meaning
playbook, stage, action	Identifier of the attack action in the playbook
attack_id	ATT&CK technique or sub-technique identifier
primary_leaf	First-failed control-point leaf used for primary attribution
primary_domain	Top-level control-point domain of the primary leaf
canonical_phase	Standard attack phase used for phase-aware scoring
primary_defense	Product categories expected to directly address the weakness
compensating_defense	Product categories that may provide secondary detection or mitigation
confidence	Confidence of the primary attribution
stage_logic	Aggregation semantics used later at the stage level

The opportunity map also records the aggregation semantics used for each playbook stage. These semantics specify how the action-level hit values within a stage should be combined during scoring. For example, a stage may represent alternative attack paths, sequentially required actions, or a loose cluster of related actions. The detailed aggregation rules are defined in the scoring model.

Table 3 summarizes the main fields used in the defense opportunity map.

Through these steps, benchmark construction turns attributed attack actions into structured defense opportunities. The resulting records connect four pieces of information required for scoring: what the attacker did, which victim-side control point first failed, which defense categories were expected to cover the action, and how the action should contribute to stage-level aggregation.

4.4 Defense Result Alignment

Defense scoring starts by aligning the evaluated system’s observations with the benchmark records. Each observation describes how the system responded to a specific playbook action. The alignment step matches the observation to the corresponding defense opportunity record and then converts the observed result into a normalized hit value.

We represent the defense observation trace as a set of action-level records. Each record contains the playbook identifier, the stage number, the action field used in the benchmark, and the observed result. Each defense observation is matched to the action record in the defense opportunity map with the same playbook identifier, stage number, and action field. After alignment, the observed result is associated with the action’s control-point attribution, canonical phase, expected defense categories, confidence label, and stage aggregation semantics.

Actions without a matched defense observation are treated as missed actions. Under this rule, an action receives a positive hit value only when the evaluated defense system reports that it was prevented, blocked, or detected. Otherwise, the action is treated as

Table 4: Defense observation results and hit values.

Result	Hit	Meaning
PREVENTED	1.0	The attack path is architecturally removed before the action can occur.
BLOCKED	0.9	The action is attempted but synchronously interrupted by the defense system.
DETECTED	0.5	The action is observed or alerted, but not stopped.
MISSED	0.0	The action is neither prevented, blocked, nor detected.

MISSED. It also keeps the benchmark comparable across different defense systems: every system is evaluated against the same set of benchmark actions, and missing observations are interpreted as lack of demonstrated coverage.

Each matched observation is then converted into one of four hit levels. PREVENTED denotes architectural prevention, where the attack path is removed before the action can be attempted. BLOCKED denotes real-time interruption, where the action is attempted but stopped by the defense system. DETECTED denotes successful observation or alerting without stopping the action itself. MISSED denotes the absence of prevention, blocking, or detection. These levels are mapped to numerical hit values used by the scoring model.

The distinction between PREVENTED and BLOCKED is important. Prevention means that the attack path is unavailable by design, such as when an egress policy makes an unauthorized destination unreachable. Blocking means that the action is observed and interrupted during execution, such as when an endpoint product terminates a malicious process. Both are stronger than detection-only outcomes, but prevention is assigned a slightly higher hit value because it does not depend on runtime recognition of the specific attack instance.

This alignment step produces a unified action-level scoring input. Each benchmark action is associated with both an expected defense opportunity and an observed defense result.

4.5 Stage-Aware Scoring Model

After defense result alignment, each benchmark action has an action-level hit value and a set of benchmark metadata, including its canonical phase, attribution confidence, and stage aggregation semantics. The scoring model uses these fields to compute defense scores at three levels: action, stage, and playbook.

The main design principle is to separate defense quality from stage importance. The stage score measures how well the defense performed within a stage, while the stage weight determines how much that stage contributes to the playbook-level score. Let a denote an attack action. Its hit value $h(a) \in [0, 1]$ is obtained from the aligned defense result described in the previous subsection.

The action weight $w(a)$ is defined as

$$w(a) = W_{\text{phase}}(\phi(a)) \cdot C_{\text{conf}}(\gamma(a)),$$

where $\phi(a)$ is the canonical phase assigned to action a , W_{phase} is the phase-weight function, $\gamma(a)$ is the attribution confidence label, and

C_{conf} is the confidence factor. In our benchmark, the confidence factor reduces the contribution of actions whose attribution is less certain. For example, high-, medium-, and low-confidence actions can be assigned factors of 1.0, 0.8, and 0.5, respectively.

For a playbook stage s , let A_s be the set of actions in that stage. The stage score S_s is computed according to the aggregation semantics of the stage. We use three aggregation types: OR, AND, and Cluster.

For an OR stage, the attacker only needs one action in the stage to succeed in order to proceed. Therefore, the defense must block all actions in the stage. The stage score is defined as

$$S_s = \min_{a \in A_s} h(a).$$

This reflects the weakest-link constraint: if any action is missed, the stage is considered poorly defended.

For an AND stage, the attacker needs all actions in the stage to succeed in order to complete the stage objective. Therefore, blocking any one required action can disrupt the stage. The stage score is defined as

$$S_s = \max_{a \in A_s} h(a).$$

This reflects the strongest defensive interruption within the stage.

For a Cluster stage, the actions do not have a strong logical dependency. They are treated as a group of related behaviors rather than as strict alternatives or strict requirements. The stage score is computed as a weighted average:

$$S_s = \frac{\sum_{a \in A_s} h(a)w(a)}{\sum_{a \in A_s} w(a)}.$$

In this case, actions with higher phase importance or higher attribution confidence contribute more to the stage score.

The stage weight is computed separately from the stage score:

$$W_s = \sum_{a \in A_s} w(a).$$

This separation avoids mixing the quality of defense within a stage with the importance of that stage in the overall playbook. In particular, OR and AND stage scores are not weighted internally, because their semantics are determined by the attacker's path structure. The weights are used when aggregating stages into a playbook score.

For a playbook P with stages $S(P)$, the playbook-level score is

$$\text{Score}(P) = \frac{\sum_{s \in S(P)} W_s S_s}{\sum_{s \in S(P)} W_s}.$$

The overall defense capability score is then computed by aggregating the scores over all evaluated playbooks:

$$\text{Score}_{\text{overall}} = \frac{1}{|\mathcal{P}|} \sum_{P \in \mathcal{P}} \text{Score}(P),$$

where $\mathcal{P}$ denotes the set of playbooks in the benchmark.

This formulation keeps the final score traceable: a score loss can be traced back from the overall score to a playbook, then to a stage, and finally to the individual actions and control-point failures that caused the loss.

4.6 Multi-Dimensional Score Outputs

The overall score is useful, but it is only a starting point. Two defense systems may receive similar aggregate scores for different reasons: one may detect many actions late, while another may cover fewer actions but block them reliably within its intended scope. For this reason, the framework reports scores from multiple perspectives: overall performance, attack phase, control-point domain, defense product category, and action-level attribution.

The phase-wise score groups results by canonical attack phase. This view answers the question: at which attack phases does the defense system lose the most score? Since the scoring model assigns higher value to earlier interception, phase-wise results help distinguish early-stage blocking from late-stage observation. This directly supports stage-aware analysis of APT defense capability.

The control-point domain score groups results by the primary victim-side control-point domain. This view answers a different question: which type of defensive control fails most often or contributes most to score loss? For example, a low score in an identity-related domain points to weaknesses in credential and trust controls, while a low score in an observability-related domain indicates insufficient detection or response. This decomposition turns an aggregate score into repair-oriented feedback.

The product category score evaluates performance within the expected coverage of each defense product category. For a product category c , let A_c denote the set of actions whose defense opportunity records include c as a primary or compensating defense category. The coverage of c is the fraction of benchmark actions that fall into this expected coverage set:

$$\text{Coverage}(c) = \frac{|A_c|}{|\mathcal{A}|},$$

where $\mathcal{A}$ is the set of benchmark actions.

The hit rate of c is computed only within its coverage set:

$$\text{HitRate}(c) = \frac{\sum_{a \in A_c} \tilde{h}_c(a)}{|A_c|}.$$

Here, $\tilde{h}_c(a)$ is the hit value credited to category c . Primary defense matches receive full hit credit, while compensating defense matches may receive discounted credit because they detect or mitigate a weakness but do not directly eliminate the failed control point.

Reporting both coverage and hit rate is important. Coverage describes how much of the benchmark a product category is expected to address. Hit rate describes how well it performs within that expected scope. A product with narrow coverage but high hit rate has a different defense profile from a product with broad coverage but low hit rate, even if their aggregate scores are similar.

Finally, the framework produces an action-level attribution report. For each score loss, the report retains the playbook, stage, action, ATT&CK technique, canonical phase, primary control-point leaf, defense categories, and observed result. This report provides the trace from aggregate score back to concrete attack actions and failed control points. As a result, the output can support both high-level comparison and detailed remediation analysis.

4.7 Prototype Method Validation

The preceding subsections define the attribution schema, benchmark construction process, result alignment procedure, scoring

Table 5: Prototype validation of defensive weakness attribution.

Field	Agreement	κ	Interpretation
Scope	100.0%	1.000	Almost perfect
L1 domain	96.0%	0.953	Almost perfect
L4 leaf	86.0%	0.856	Almost perfect
Confidence	82.0%	0.679	Substantial
Mapping method	82.0%	0.687	Substantial
Secondary tag	66.0%	0.173	Slight

model, and multi-dimensional outputs. Before applying the framework to real defense-system replay experiments, we perform a set of prototype validation experiments on the benchmark and scoring infrastructure. These experiments test whether the proposed methodological components are reproducible, whether the control-point-to-defense mapping is reasonable, and whether the scoring engine behaves correctly under boundary and parameter-variation cases.

First, we evaluate the reproducibility of defensive weakness attribution through a pilot inter-rater reliability study. A stratified sample of 50 ATT&CK techniques and sub-techniques was independently labeled by two annotators using the same coding guide. Agreement was measured at multiple levels, including scope decision, top-level domain, leaf-level attribution, confidence label, mapping method, and secondary weakness tag. Table 5 summarizes the results.

The result shows that the primary attribution levels are reproducible in the pilot setting. The high agreement at the L1 and L4 levels indicates that the main control-point attribution can be applied consistently. In contrast, the secondary tag has much lower agreement, so it is retained as qualitative chain-context information rather than as a primary quantitative scoring field.

Second, we validate the defense-category mapping used by the Defense-Tech Crosswalk. The external validity check compares the crosswalk categories with the dataset’s native control labels. Among 1,733 comparable actions, the two sources agree on 1,297 actions, giving an agreement rate of 74.8%. We also perform an internal consistency check in which an independent annotator maps a stratified sample of 35 leaves to the 29 defense categories without seeing the existing crosswalk. Table 6 summarizes these checks.

These results suggest that the primary defense mapping is stable enough to support product-category scoring. The lower consistency for compensating defenses indicates that compensating coverage is less sharply defined than primary coverage. Therefore, compensating matches are treated as partial credit and should be interpreted more cautiously.

Third, we check the mathematical behavior of the scoring model using synthetic coverage traces. These traces are not real defense experiments; they are boundary and coverage simulations. A perfect-defense trace assigns PREVENTED to every benchmark action, and a no-defense trace assigns MISSED to every action. Two idealized

Table 6: Prototype validation of defense-category mapping.

Validation item	Result
External agreement with native control labels	74.8% (1,297 / 1,733 actions)
Highest category agreement	DLP 96.4%, NET 93.6%, EGW 90.3%
Endpoint-category agreement	AV 73.5%, EDR 71.9%
Primary defense internal agreement	97.1% (34 / 35 leaves), $\kappa = 0.485$
Compensating defense internal agreement	45.7% (16 / 35 leaves), $\kappa = -0.094$

Table 7: Prototype scoring checks with synthetic coverage traces.

Simulation	Configuration	Score
Perfect	All actions are PREVENTED	100.00
None	All actions are MISSED	0.00
Ideal EDR	EDR primary coverage → BLOCKED	44.40
Ideal NGFW	NGFW/NET primary coverage → BLOCKED	24.93

Table 8: Sensitivity and aggregation robustness checks.

Check	Result
Perfect / none under all parameter settings	100 / 0 in all configurations
EDR score range	43.41–46.67
NGFW score range	23.62–27.57
EDR/NGFW ratio range	1.57–1.94
Discovery-dominated Cluster alternative	Maximum score delta 1.07
Action-count weighted playbook aggregation	Maximum score delta 0.78

product-category traces simulate systems that block all actions in their expected EDR or NGFW/NET primary coverage. Table 7 reports the resulting scores.

The perfect and none traces verify the expected upper and lower bounds of the scoring engine. The ideal EDR and ideal NGFW simulations show that a single product category cannot cover the whole benchmark even when it performs perfectly within its expected scope. This result reflects the multi-stage and multi-control nature of APT defense rather than the performance of a specific commercial product.

Finally, we test the robustness of the scoring results under reasonable parameter and aggregation variations. We vary phase weights, confidence factors, and compensating-defense discounts, and we also compare alternative aggregation choices for discovery-heavy Cluster stages and playbook-level aggregation. Table 8 summarizes the main results.

The sensitivity analysis shows that the boundary behavior of the scoring model remains stable across parameter settings, and

the relative ordering between the idealized EDR and NGFW traces does not change. The aggregation robustness checks also show limited score variation under alternative but reasonable aggregation choices. These results support the use of the proposed scoring model as a stable prototype benchmark infrastructure. The later evaluation section will use real defense-system observations to test how deployed defenses perform under this methodology.

5 Evaluation

6 Discussion

7 Conclusion

References

- [1] Abdullallah Alsaheel, Yuhong Nan, Shiqing Ma, Le Yu, Gregory Walkup, Z. Berkay Celik, Xiangyu Zhang, and Dongyan Xu. 2021. ATLAS: A Sequence-Based Learning Approach for Attack Investigation. In *Proceedings of the 30th USENIX Security Symposium*. 3005–3022.
- [2] Martin Casado, Tal Garfinkel, Aditya Akella, Michael J. Freedman, Dan Boneh, Nick McKeown, and Scott Shenker. 2006. SANE: A Protection Architecture for Enterprise Networks. In *Proceedings of the 15th USENIX Security Symposium*.
- [3] Zijun Cheng, Qiuqian Lv, Jinyuan Liang, Yan Wang, Degang Sun, Thomas Pasquier, and Xueyuan Han. 2024. KAIROS: Practical Intrusion Detection and Investigation using Whole-System Provenance. In *Proceedings of the IEEE Symposium on Security and Privacy*. 3533–3551.
- [4] Mohan Dhawan, Rishabh Poddar, Kshiteej Mahajan, and Vijay Mann. 2015. SPHINX: Detecting Security Attacks in Software-Defined Networks. In *Network and Distributed System Security Symposium (NDSS)*. <https://www.ndss-symposium.org/ndss2015/ndss-2015-programme/sphinx-detecting-security-attacks-software-defined-networks/>
- [5] Feng Dong, Liu Wang, Xu Nie, Fei Shao, Haoyu Wang, Ding Li, Xiapu Luo, and Xusheng Xiao. 2023. DISTDET: A Cost-Effective Distributed Cyber Threat Detection System. In *Proceedings of the 32nd USENIX Security Symposium*. 6575–6592.
- [6] Min Du, Feifei Li, Guineng Zheng, and Vivek Srikumar. 2017. DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*.
- [7] Akul Goyal, Gang Wang, and Adam Bates. 2024. R-CAID: Embedding Root Cause Analysis within Provenance-Based Intrusion Detection. In *Proceedings of the IEEE Symposium on Security and Privacy*.
- [8] Xueyuan Han, Thomas Pasquier, Adam Bates, James Mickens, and Margo Seltzer. 2020. UNICORN: Runtime Provenance-Based Detector for Advanced Persistent Threats. In *Proceedings of the Network and Distributed System Security Symposium*.
- [9] Wajih Ul Hassan, Adam Bates, and Daniel Marino. 2020. Tactical Provenance Analysis for Endpoint Detection and Response Systems. In *Proceedings of the IEEE Symposium on Security and Privacy*.
- [10] Wajih Ul Hassan, Shengjian Guo, Ding Li, Zhengzhang Chen, Kangkook Jee, Zhichun Li, and Adam Bates. 2019. NoDoze: Combatting Threat Alert Fatigue with Automated Provenance Triage. In *Proceedings of the Network and Distributed System Security Symposium*.
- [11] Wajih Ul Hassan, Mark Lemay, Nuraini Aguse, Adam Bates, and Thomas Moyer. 2020. OmegaLog: High-Fidelity Attack Investigation via Transparent Multi-layer Log Analysis. In *Proceedings of the Network and Distributed System Security Symposium*.
- [12] Cormac Herley and Paul C. van Oorschot. 2017. SoK: Science, Security, and the Elusive Goal of Security as a Scientific Pursuit. In *Proceedings of the IEEE Symposium on Security and Privacy*.
- [13] Md Nahid Hossain, Sadegh M. Milajerdi, Junao Wang, Birhanu Eshete, Rigel Gjomemo, R. Sekar, Scott Stoller, and V. N. Venkatakrishnan. 2017. SLEUTH: Real-time Attack Scenario Reconstruction from COTS Audit Data. In *Proceedings of the 26th USENIX Security Symposium*. 487–504.
- [14] Zian Jia, Yun Xiong, Yuhong Nan, Yao Zhang, Jinjing Zhao, and Mi Wen. 2024. MAGIC: Detecting Advanced Persistent Threats via Masked Graph Representation Learning. In *33rd USENIX Security Symposium (USENIX Security 24)*. 5197–5214. <https://www.usenix.org/conference/usenixsecurity24/presentation/jia-zian>
- [15] Seungsoo Lee, Jinwoo Kim, and Seungwon Shin. 2017. DELTA: A Security Assessment Framework for Software-Defined Networks. In *Network and Distributed System Security Symposium (NDSS)*. <https://www.ndss-symposium.org/ndss2017/ndss-2017-programme/delta-security-assessment-framework-software-defined-networks/>

- [16] Shaofei Li, Feng Dong, Xusheng Xiao, Haoyu Wang, Fei Shao, Jiedong Chen, Yao Guo, Xiangqun Chen, and Ding Li. 2024. NODLINK: An Online System for Fine-Grained APT Attack Detection and Investigation. In *Network and Distributed System Security Symposium (NDSS)*. doi:10.14722/ndss.2024.23204
- [17] Vector Guo Li, Matthew Dunn, Paul Pearce, Damon McCoy, Geoffrey M. Voelker, and Stefan Savage. 2019. Reading the Tea Leaves: A Comparative Analysis of Threat Intelligence. In *28th USENIX Security Symposium (USENIX Security 19)*. <https://www.usenix.org/conference/usenixsecurity19/presentation/li-vector>
- [18] Xiaojing Liao, Kan Yuan, XiaoFeng Wang, Zhou Li, Luyi Xing, and Raheem Beyah. 2016. Acing the IoC Game: Toward Automatic Discovery and Analysis of Open-Source Cyber Threat Intelligence. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. 755–766. doi:10.1145/2976749.2978315
- [19] Fucheng Liu, Yu Wen, Dongrui Zhang, Xinyu Jiang, Xinyu Xing, and Dan Meng. 2019. Log2Vec: A Heterogeneous Graph Embedding Based Approach for Detecting Cyber Threats within Enterprise. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*.
- [20] Zaoxing Liu, Hun Namkung, Georgios Nikolaidis, Jeongkeun Lee, Changhoon Kim, Xin Jin, Vladimir Braverman, Minlan Yu, and Vyas Sekar. 2021. Jaqen: A High-Performance Switch-Native Approach for Detecting and Mitigating Volumetric DDoS Attacks with Programmable Switches. In *30th USENIX Security Symposium (USENIX Security 21)*. 3829–3846. <https://www.usenix.org/conference/usenixsecurity21/presentation/liu-zaoxing>
- [21] Sadeq M. Milajerdi, Birhanu Eshete, Rigel Gjomo, R. Sekar, and V. N. Venkatakrishnan. 2019. POIROT: Aligning Attack Behavior with Kernel Audit Records for Cyber Threat Hunting. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*.
- [22] Sadeq M. Milajerdi, Rigel Gjomo, Birhanu Eshete, R. Sekar, and V. N. Venkatakrishnan. 2019. HOLMES: Real-time APT Detection through Correlation of Suspicious Information Flows. In *Proceedings of the IEEE Symposium on Security and Privacy*.
- [23] Mati Ur Rehman, Hadi Ahmadi, and Wajih Ul Hassan. 2024. FLASH: A Comprehensive Approach to Intrusion Detection via Provenance Graph Representation Learning. In *2024 IEEE Symposium on Security and Privacy*. 3552–3570. doi:10.1109/SP54263.2024.00139
- [24] Yun Shen and Gianluca Stringhini. 2019. ATTACK2VEC: Leveraging Temporal Word Embeddings to Understand the Evolution of Cyberattacks. In *28th USENIX Security Symposium (USENIX Security 19)*. 905–921. <https://www.usenix.org/conference/usenixsecurity19/presentation/shen>
- [25] Oleg Sheyner, Joshua Haines, Somesh Jha, Richard Lippmann, and Jeannette M. Wing. 2002. Automated Generation and Analysis of Attack Graphs. In *Proceedings of the IEEE Symposium on Security and Privacy*. 273–284.
- [26] Seungwon Shin, Vinod Yegneswaran, Phillip Porras, and Guofei Gu. 2013. AVANT-GUARD: Scalable and Vigilant Switch Flow Management in Software-Defined Networks. In *Proceedings of the 2013 ACM SIGSAC Conference on Computer and Communications Security*. doi:10.1145/2508859.2516684
- [27] Lingyu Wang, Sushil Jajodia, Anoop Singhal, Pengsu Cheng, and Steven Noel. 2014. k-Zero Day Safety: A Network Security Metric for Measuring the Risk of Unknown Vulnerabilities. *IEEE Transactions on Dependable and Secure Computing* 11, 1 (2014), 30–44.
- [28] Su Wang, Zhiliang Wang, Tao Zhou, Xia Yin, Dongqi Han, Han Zhang, Hongbin Sun, Xingang Shi, and Jiahai Yang. 2022. threaTrace: Detecting and Tracing Host-Based Threats in Node Level through Provenance Graph Learning. *IEEE Transactions on Information Forensics and Security* 17 (2022), 3972–3987.
- [29] Jiarong Xing, Wenqing Wu, and Ang Chen. 2021. Ripple: A Programmable, Decentralized Link-Flooding Defense Against Adaptive Adversaries. In *30th USENIX Security Symposium (USENIX Security 21)*. 3865–3880. <https://www.usenix.org/conference/usenixsecurity21/presentation/xing>
- [30] Fan Yang, Jiachen Xu, Chunlin Xiong, Zhou Li, and Kehuan Zhang. 2023. PRO-GRAPHER: An Anomaly Detection System based on Provenance Graph Embedding. In *32nd USENIX Security Symposium (USENIX Security 23)*. 4355–4372. <https://www.usenix.org/conference/usenixsecurity23/presentation/yang-fan>
- [31] Sophia Yoo, Xiaoqi Chen, and Jennifer Rexford. 2024. SmartCookie: Blocking Large-Scale SYN Floods with a Split-Proxy Defense on Programmable Data Planes. In *33rd USENIX Security Symposium (USENIX Security 24)*. 217–234. <https://www.usenix.org/conference/usenixsecurity24/presentation/yoo>
- [32] Lihua Yuan, Hao Chen, Jianning Mai, Chen-Nee Chuah, Zhendong Su, and Prasant Mohapatra. 2006. FIREMAN: A Toolkit for Firewall Modeling and Analysis. In *Proceedings of the IEEE Symposium on Security and Privacy*. 199–213.
- [33] Bo Zhang, Yansong Gao, Changlong Yu, Boyu Kuang, Zhi Zhang, Hyoungshick Kim, and Anmin Fu. 2025. TAPAS: An Efficient Online APT Detection with Task-guided Process Provenance Graph Segmentation and Analysis. In *34th USENIX Security Symposium (USENIX Security 25)*. 607–624. <https://www.usenix.org/conference/usenixsecurity25/presentation/zhang-bo-tapas>
- [34] Menghao Zhang, Guanyu Li, Shicheng Wang, Chang Liu, Ang Chen, Hongxin Hu, Guofei Gu, Qi Li, Mingwei Xu, and Jianping Wu. 2020. Poseidon: Mitigating Volumetric DDoS Attacks with Programmable Switches. In *Network and Distributed System Security Symposium (NDSS)*. <https://www.ndss-symposium.org/ndss-paper/poseidon-mitigating-volumetric-ddos-attacks-with-programmable-switches/>
- [35] Mengyuan Zhang, Lingyu Wang, Sushil Jajodia, Anoop Singhal, and Massimiliano Albanese. 2016. Network Diversity: A Security Metric for Evaluating the Resilience of Networks Against Zero-Day Attacks. *IEEE Transactions on Information Forensics and Security* 11, 5 (2016), 1071–1086.

A Appendix

Received 7 October 2025; revised 24 December 2025; accepted 13 January 2026